

# Characterizing Vision-Language-Action Models across XPU: Constraints and Acceleration for On-Robot Deployment

VLA 模型跨 XPU 表征：面向机器人部署的约束与加速

Kaijun Zhou, Qiwei Chen, Da Peng, Zhiyang Li, Xijun Li, Jinyu Gu

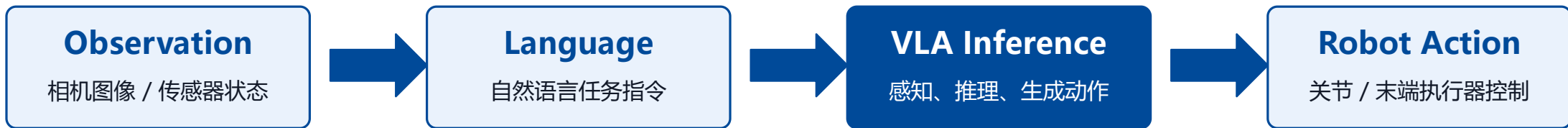
School of Computer Science, Shanghai Jiao Tong University · ICML; 2026



|   |         |
|---|---------|
| 1 | 研究背景    |
| 2 | 模型-硬件配对 |
| 3 | 计算特征分析  |
| 4 | 加速方法    |
| 5 | 小结      |

|   |         |
|---|---------|
| 1 | 研究背景    |
| 2 | 模型-硬件配对 |
| 3 | 计算特征分析  |
| 4 | 加速方法    |
| 5 | 小结      |

## Vision-Language-Action: 把视觉、语言和机器人动作连接成闭环控制



### 部署难点

机器人控制是 observe-infer-act 闭环；推理慢会带来停顿、振荡和任务失败。

### 本文视角

从模型与硬件共同表征出发，关注真实端到端延迟、能耗和成本，而非只看模型成功率。

VLA 上机器人部署首先是系统问题：实时性、成本和能耗必须一起考虑。

# 研究背景

既然 **VLA** 部署是 **observe-infer-act** 闭环系统，选硬件时第一步就不是“谁峰值算力最高”，而是“谁能先满足实时控制”

先判断某块硬件能不能达到任务要求的控制频率；达不到，就直接淘汰。

## ① 先定门槛

先明确任务需要的最低控制频率 / 最大延迟，  
例如： $\geq 5 \text{ Hz}$ ，或单次推理  $\leq 200 \text{ ms}$ 。

先把目标讲清楚

## ② 先筛 Time

检查每块硬件能否达到这个门槛；  
达不到就直接排除，因为会造成闭环滞后、动作不稳甚至任务失败。

Time = 能不能用

## ③ 再比 Cost / Energy

**Cost**  
每台机器人都要配算力  
规模化时成本敏感

**Energy**  
电池容量与散热受限  
移动机器人尤为关键

值不值得用

只在“已通过实时性筛选”的候选硬件里比较这两项。

## VLA 模型结构正在从单体动作 token 生成, 转向 VLM + Action Expert 双系统

### 单体 / 动作 Token

代表: OpenVLA、RT-2 等

- VLM 输出离散 action tokens
- 类似语言模型自回归 decoding
- 延迟受 prefill + decode 共同影响

image + instruction  $\rightarrow$  token1  $\rightarrow$  token2  $\rightarrow$  action

### 双系统 / Action Expert

代表:  $\pi$ 0、 $\pi$ 0.5、Gr00t、SmolVLA

- VLM 负责语义与视觉特征
- Action Expert 负责连续动作
- diffusion / flow matching 通常需要多步迭代

image + instruction  $\rightarrow$  VLM features  $\rightarrow$  AE steps  $\rightarrow$  action

本文的关键发现主要来自第二类双系统 VLA: 两阶段瓶颈和两层串行化。

回答：VLA 真正上机器人时，怎么选硬件、找瓶颈、做加速？



**VLA 推理是由性质不同的阶段串联而成，单看参数量或峰值 FLOPS 都会误导部署决策。**  
正确路径是 model-hardware co-characterization：先实测，再分析瓶颈，再做 phase-aware 优化。

|   |         |
|---|---------|
| 1 | 研究背景    |
| 2 | 模型-硬件配对 |
| 3 | 计算特征分析  |
| 4 | 加速方法    |
| 5 | 小结      |

# 模型-硬件配对

## 第一步：用模型规模、算力和显存容量建立可行性先验

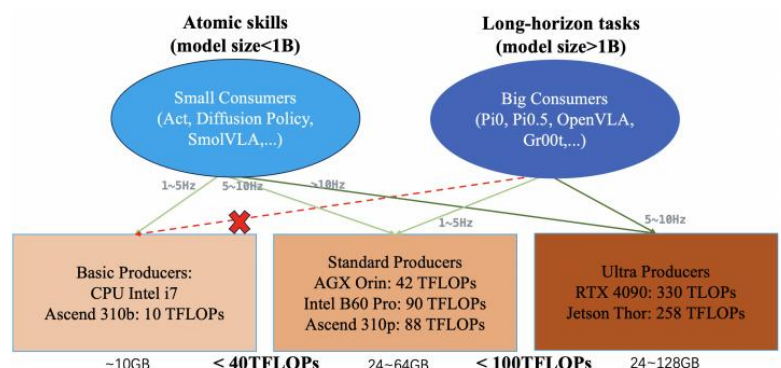


Figure 1. The diagram illustrates the compatibility matrix between VLA model types (Consumers) and hardware platforms (Producers), together with the hardware memory and peak FLOPs.

### 1 模型分层

Small Consumers: ACT、Diffusion Policy、SmoVLA  
Big Consumers: Gr00t、 $\pi$ 0、 $\pi$ 0.5、OpenVLA

### 2 硬件分层

Basic / Standard / Ultra Producers: 按 FP16/BF16 峰值算力粗分，但还要看显存容量。

硬门槛：放不下模型或达不到目标控制频率，就不进入后续成本/能耗排序。

## CET Leaderboard: Cost / Energy / Time 联合筛选模型-硬件组合

| π0 平台    | Cost   | Energy   | Time     |
|----------|--------|----------|----------|
| RTX 4090 | \$3500 | 2.398 kJ | 102.3 ms |
| Thor     | \$3400 | 1.282 kJ | 246.0 ms |
| Orin     | \$1999 | 1.866 kJ | 920.6 ms |
| B60      | \$599  | 6.363 kJ | 306.5 ms |
| 310P     | \$1030 | 2.618 kJ | 818.0 ms |

Time 是可行性门槛；Cost 和 Energy 是可行平台之间的部署权衡。

Time

控制频率 / 延迟：先判定是否可行

Cost

硬件采购成本：规模化部署敏感

Energy

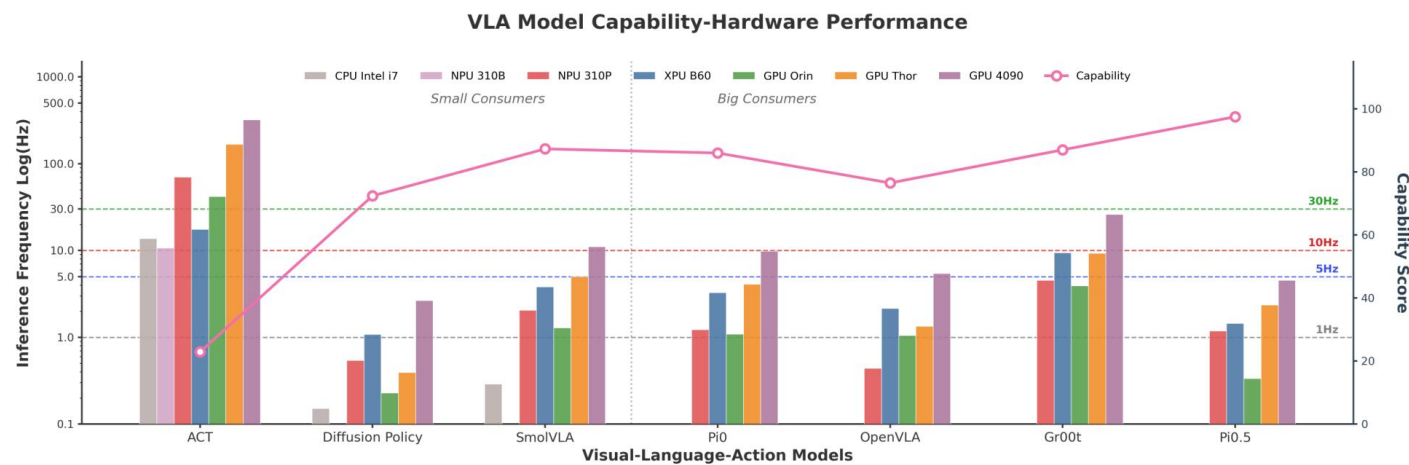
推理能耗：电池与散热约束

先筛掉不可行，再按场景权重排序

“最好硬件”不是固定的，而是随模型和部署目标改变。

Finding 1：模型参数量不是推理频率的可靠预测器

反直觉现象



小模型 Diffusion Policy 可能比更大的  $\pi_0$  更慢。

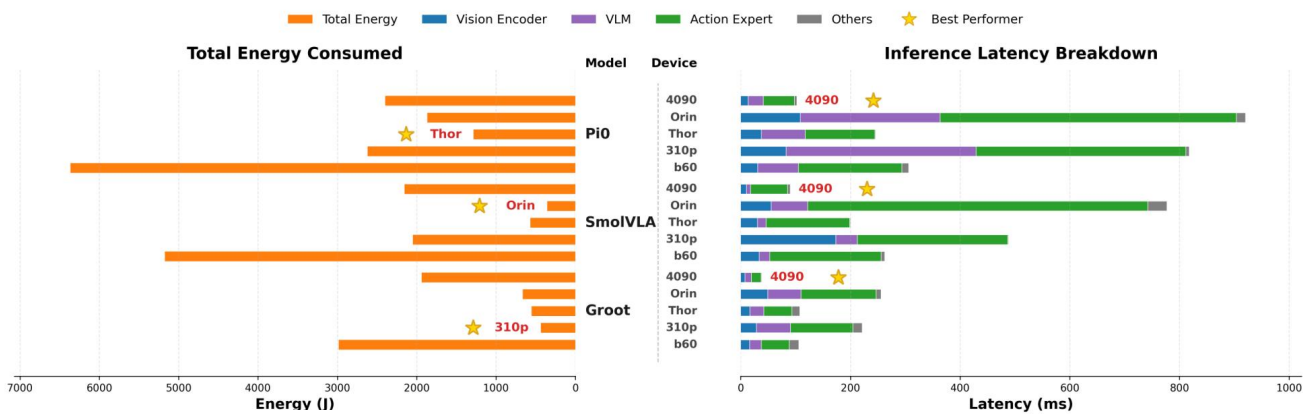
原因不是参数量，而是 Action Expert 的迭代次数： $\pi_0$  约 4 步，Diffusion Policy 可达 100 步。

Figure 2. VLA model performance across hardware accelerators. The bar chart displays inference frequencies (log scale) across hardware platforms for VLA models. The pink line represents capability scores on LIBERO (Liu et al., 2023). Horizontal lines indicate frequency thresholds (1Hz-30Hz).

总时间  $\approx$  单步时间  $\times$  迭代步数

部署评估必须看端到端控制频率，而不是只看模型大小。

Finding 2: 多维指标下，最优硬件因模型和目标而异



1 Latency priority

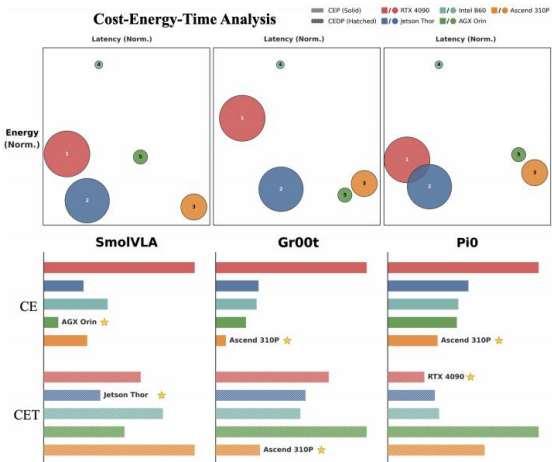
动态任务优先选择最快平台； $\pi 0$  示例中 RTX 4090 最快。

2 Energy priority

电池机器人优先选择端到端能耗低的平台；Thor 在真实任务中能效突出。

3 CE / CET 排序

把主观选型转化成“先筛可行、再按权重排序”的可复现实验流程。



基于多维指标 (CET) 的硬件选择指南。气泡大小表示成本 (Cost)，横轴表示能耗 (Energy)，纵轴表示归一化推理延迟 (Time)。

CE：在能跑的硬件里，优先挑便宜又省电的。  
CET：在能跑的硬件里，把速度、价格、耗电一起权衡

# 模型-硬件配对

## Leaderboard 的实际使用方式：三步筛选

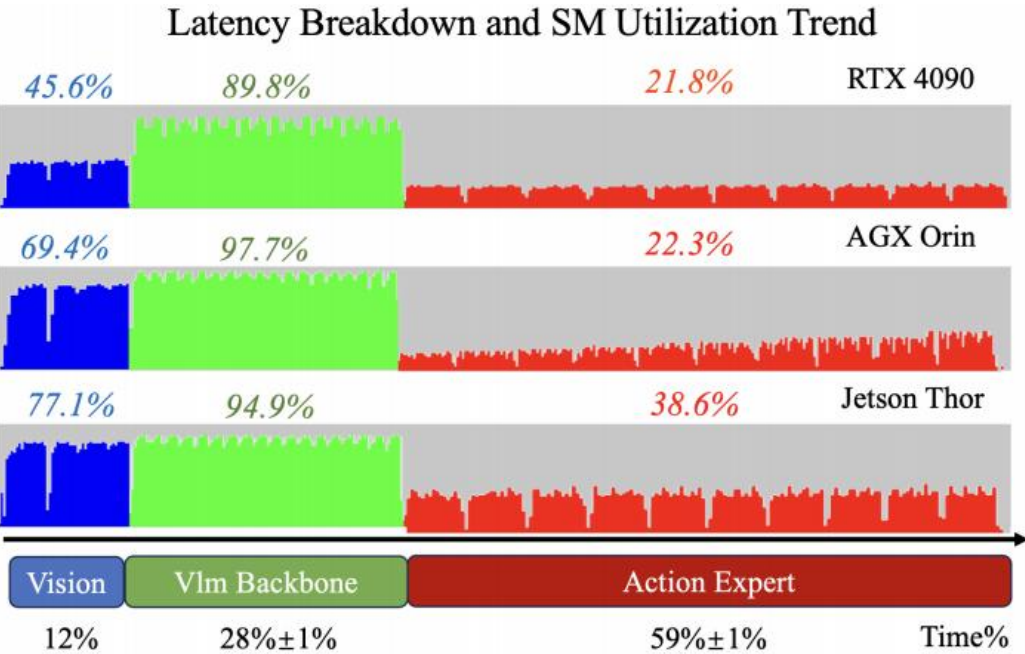


$\pi 0$  示例：追求最快选 RTX 4090；规模部署看成本可考虑 B60 / 310P；电池续航优先看 Thor / Orin。

配对评测回答了“谁能跑、谁更合适”；但还没有回答“为什么会这样慢”

|   |         |
|---|---------|
| 1 | 研究背景    |
| 2 | 模型-硬件配对 |
| 3 | 计算特征分析  |
| 4 | 加速方法    |
| 5 | 小结      |

Finding 3:  $\pi 0$  工作流存在明显的双阶段计算失衡



$\pi 0$  在 NVIDIA 平台上的性能分析结果。蓝、绿、红三种颜色表示不同推理阶段，数字表示各阶段的 SM 利用率。SM (Streaming Multiprocessor, 流式多处理器) 是 NVIDIA GPU 中执行并行计算的核心单元，SM 利用率越高，说明 GPU 计算资源使用得越充分。

VLM Backbone: SM 利用率通常 >90%，更接近满负载计算。

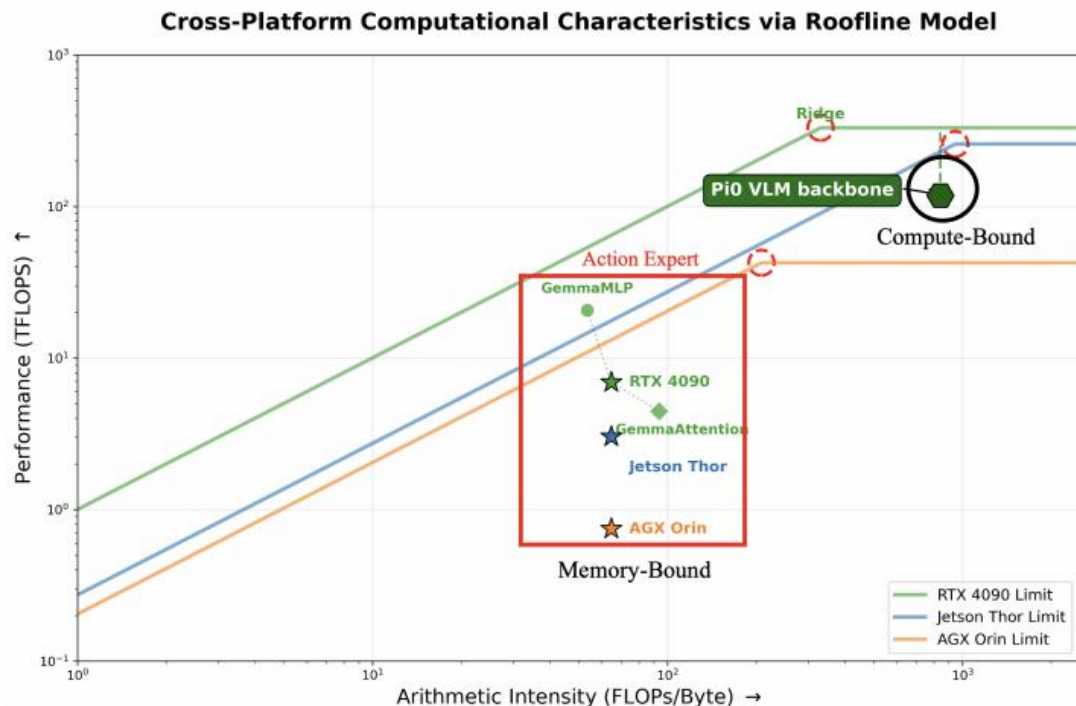
Action Expert: SM 利用率约 20%–40%，但延迟约为 VLM 的 2×。

结果：高性能计算单元在 Action Expert 阶段大量空转，端到端频率低于峰值规格预期。

VLA 不是均匀 workload：一个硬件可能适合 VLM，但不适合 AE。

# 计算特征分析

## Roofline: 用 operational intensity 判断算力瓶颈还是带宽瓶颈



$\pi_0$  计算特性的 Roofline 分析。RTX 4090、AGX Orin 和 Jetson Thor 均测试了 Action Expert 中的 GemmaDecoderLayer；RTX 4090 还进一步分析了 VLM Backbone 中的 GemmaMLP 和 GemmaAttention。

### 核心公式

Operational Intensity:  $I = \text{FLOPs} / \text{Bytes moved}$

Ridge point:  $I_r = \text{Peak FLOPS} / \text{Bandwidth}$

若  $I < I_r$ : memory-bound; 若  $I > I_r$ : compute-bound。

**Operational Intensity** (计算强度)：模型每从内存里搬 1 份数据，能做多少计算。

**Ridge point**：判断任务到底更缺算力还是更缺内存带宽的分界点。

同一模型在不同硬件上表现不同，是因为不同硬件“算力”和“内存带宽”的匹配程度不同。

# 计算特征分析

**Finding 4:** VLA 的两个阶段“卡点”正好相反:前面的 VLM 更缺算力, 后面的 Action Expert 更缺内存带宽。

## VLM Backbone

### Compute-bound

$\pi 0$  VLM DecoderLayer:  
185.1G FLOPs / 220MB  $\approx$  840 FLOPs/Byte  
高于 RTX 4090 ridge point 330 FLOPs/Byte

优化方向: 减少算术量、编译、量化、kernel fusion

## Action Expert

### Memory-bound

$\pi 0$  AE:  $I \approx 64.5$  FLOPs/Byte  
远低于 4090 / Thor / Orin 的 ridge point  
多步 diffusion / flow 迭代进一步放大带宽瓶颈

优化方向: 减少迭代、复用中间结果、提高数据复用

VLM 和 AE 的瓶颈性质相反, 因此必须 phase-aware 优化。

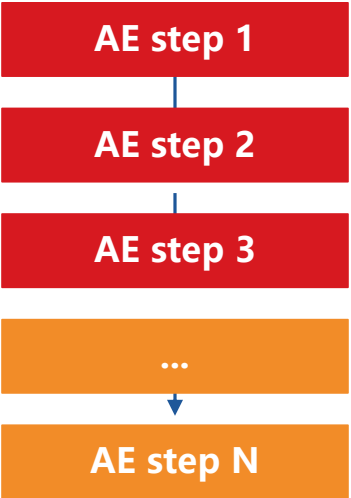
# 计算特征分析

## VLA 延迟的本质：两层串行化

### Pipeline-level serialization



### Component-level serialization



### 原始总延迟

总延迟 = 视觉处理 + VLM 推理 + N 次 Action Expert 迭代

1. 降低单步开销

2. 减少有效迭代次数

3. 重叠串行阶段

|   |         |
|---|---------|
| 1 | 研究背景    |
| 2 | 模型-硬件配对 |
| 3 | 计算特征分析  |
| 4 | 加速方法    |
| 5 | 小结      |

先看已有加速技术的组合：OpenVLA 上并非“收益简单相加”

| Method         | Speedup      | Freq. (Hz)  | Spatial (%) | Object (%) | Goal (%) | Long (%) | Avg (%) |
|----------------|--------------|-------------|-------------|------------|----------|----------|---------|
| Baseline       | 1.00×        | 5.41        | 84.7        | 88.4       | 79.2     | 53.7     | 76.5    |
| Quant. (4-bit) | 1.14×        | 6.17        | 77.4        | 64         | 75.8     | 53.4     | 67.7    |
| Cache          | 1.16×        | 6.29        | 83.8        | 85.8       | 76.4     | 52.8     | 74.7    |
| Spec.          | 1.11×        | 6.02        | 85.8        | 85.0       | 74.4     | 55.0     | 75.1    |
| Spec. + Cache  | <b>1.29×</b> | <b>6.99</b> | 78.6        | 71.4       | 76       | 48       | 68.5    |
| Quant. + Cache | 1.27×        | 6.84        | 76.6        | 62.2       | 72.4     | 49       | 65.1    |

Cache 与 Speculative 作用于不同阶段，但叠加后成功率可能明显下降。

Quantization 会破坏 speculative 的数值一致性，降低 draft 接受率。

因此必须同时看端到端速度和成功率，不能只看单项 speedup。

Cache 优化输入处理阶段

Speculative 优化动作 token 生成阶段，两者解决的是不同瓶颈

多个优化必须从端到端速度和成功率同时评估，不能只看单项 speedup。

模型编译：减少单步执行开销，但收益随硬件和软件栈变化

Table 3. Performance acceleration of  $\pi_0$  using compilation across different hardware platforms.

| Hardware    | Baseline (ms) | Compiled (ms) | Speedup (×)  | Freq. (Hz) |
|-------------|---------------|---------------|--------------|------------|
| RTX 4090    | 102.3         | 35.2          | <b>2.90×</b> | 28.41      |
| Jetson Thor | 246.0         | 163.0         | <b>1.51×</b> | 6.13       |
| Ascend 310P | 818.0         | 350.0         | <b>2.34×</b> | 2.86       |

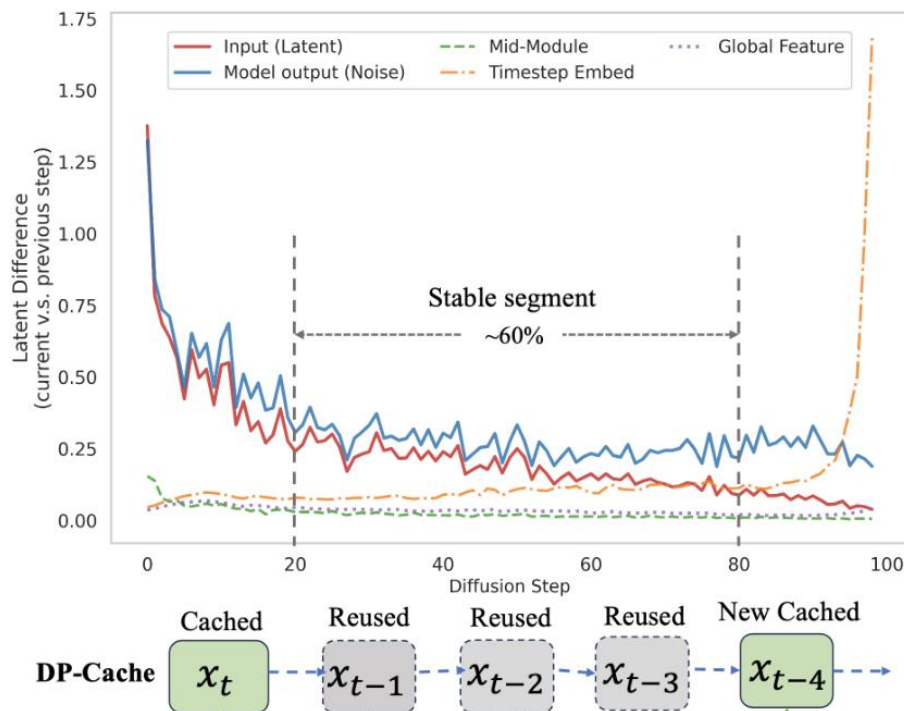
NVIDIA: torch.compile / JIT / kernel fusion  
Ascend: ATC 静态图编译 / offline inference

同一  $\pi_0$  编译加速

RTX 4090: 102.3 → 35.2 ms, 2.90×  
Jetson Thor: 246 → 163 ms, 1.51×  
Ascend 310P: 818 → 350 ms, 2.34×

结论：compiler 不是硬件无关的技术；硬件后端实现决定可获得收益。

## DP-Cache: 利用 diffusion 中段的时间冗余，减少 Action Expert 实际计算次数



### 核心机制

1. 先离线 profiling diffusion trajectory
2. 找到相邻 step 差异较小的 stable segment
3. 在稳定窗口内周期性计算一次，其余 step 复用缓存中间结果

compute → reuse → reuse → reuse →  
compute

DP-Cache 的作用，就是让 Action Expert 在多轮迭代时不用每一步都重新算，能复用的中间结果就直接复用，从而减少实际计算次数。

DP-Cache 效果：速度提升与成功率损失之间存在权衡

Table 4. DP-Cache: Inference Latency and Capability (SR) on RTX 4090.

| Method             | Speedup | Latency (ms) | Freq. (Hz) | Spatial (%) | Object (%) | Long (%) |
|--------------------|---------|--------------|------------|-------------|------------|----------|
| Baseline (DP)      | 1.00×   | 378          | 2.6        | 75.4        | 52.4       | 74.1     |
| DP-Cache ( $S=4$ ) | 1.89×   | 200          | 5.0        | 78.9        | 51.4       | 76.9     |
| DP-Cache ( $S=8$ ) | 2.09×   | 181          | 5.6        | 71.6        | 46.1       | 73.4     |

Table 5. Franka deployment: DP-Cache speedup and task success rate for  $\pi_{0.5}$ -droid (50 trials).

| Method                   | Speedup | Latency (ms) | SR (%) |
|--------------------------|---------|--------------|--------|
| Baseline ( $\pi_{0.5}$ ) | 1×      | 95           | 76     |
| DP-Cache ( $S=4$ )       | 1.28×   | 74           | 70     |

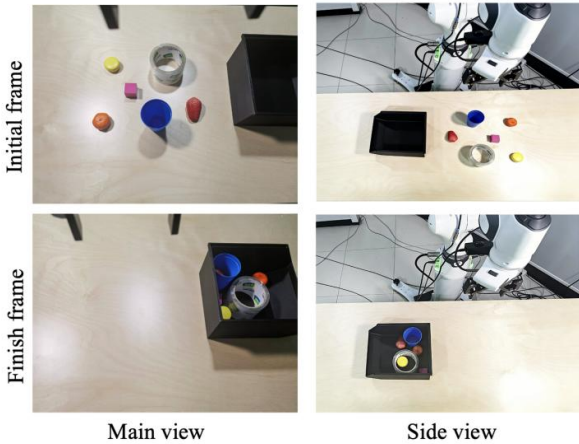


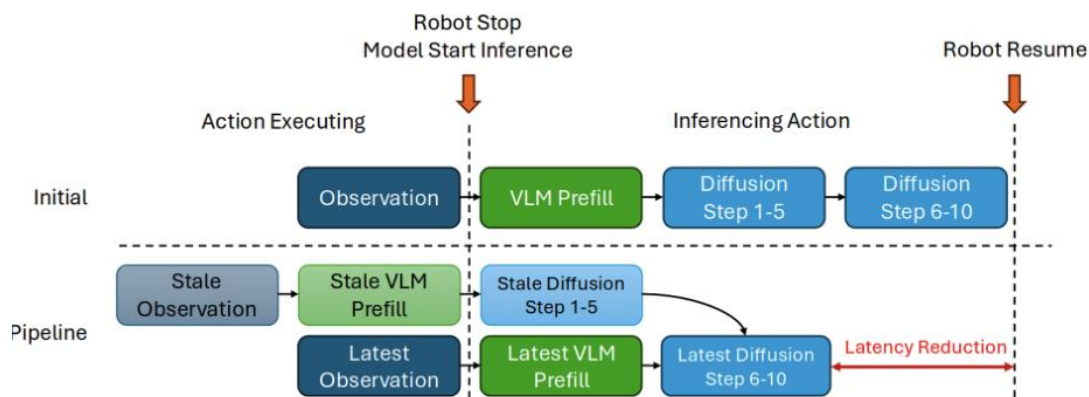
Figure 8. Visual demonstration of the manipulation task on Franka using  $\pi_{0.5}$ -droid: Cleaning up the table task.

真实 Franka 清桌任务： $\pi_{0.5}$  从 95 ms 降到 74 ms, 1.28×, SR 从 76% 降到 70%。

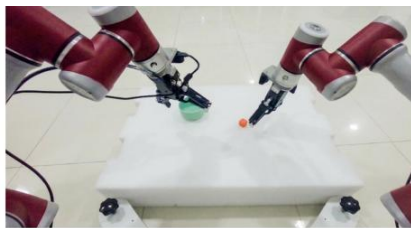
与编译结合后，Diffusion Policy 在 4090 达 2.9×，在 Ascend 310P 达 6.0×。

# 加速方法

## V-AEFusion: 利用相邻观测相似性, 把 VLM 与 AE 早期步骤重叠执行



(a) Last timestep observation



(b) Current timestep observation

当前这一帧还在做VLM推理时, Action Expert 先拿上一帧的视觉信息开始生成动作的前几步。

等当前这一帧的 VLM 结果出来后, Action Expert 立刻换成最新的视觉信息, 继续完成后面的动作细化。

因为机器人相邻两帧通常变化不大, 而且 diffusion 前几步主要决定大致怎么动, 后几步才负责把动作位置修

V-AEFusion: 让 VLM 和 Action Expert 尽量同时工作, 减少前后阶段互相等待的时间。

Stale steps 不是越多越好：并行收益与条件过时风险之间取平衡



(a) Placing Block in Bowl



(b) Placing Cup in Bowl

Stale steps: Action Expert 暂时使用“上一帧的旧视觉信息”来执行的前几步 denoising。

Table 7. Success rates (%) on two manipulation tasks as a function of the number of stale denoising steps (50 trials per task).

| Stale Steps | Putting Block in Bowl | Putting Cup in Bowl |
|-------------|-----------------------|---------------------|
| 0           | 92                    | 86                  |
| 1           | 92                    | 84                  |
| 3           | 90                    | 82                  |
| 5           | 88                    | 78                  |
| 6           | 80                    | 66                  |
| 7           | 50                    | 42                  |
| 9           | 10                    | 6                   |

Table 8. V-AEFusion: inference latency and capability comparison of  $\pi_0$  on RTX 4090.

| Method               | Speedup | Latency (ms) | Spatial (%) | Object (%) | Goal (%) | Long (%) | Avg (%) |
|----------------------|---------|--------------|-------------|------------|----------|----------|---------|
| Baseline ( $\pi_0$ ) | 1×      | 102          | 64.0        | 78.4       | 72.8     | 35.2     | 62.6    |
| V-AEFusion           | 1.3×    | 77           | 60.2        | 76.6       | 77.4     | 35.0     | 62.3    |

两个支撑假设：① 相邻闭环观测高度相似；② diffusion 早期决定粗轨迹，后期 fresh feature 可修正空间细节。  
消融结论：超过 6 个 stale steps 后成功率明显下降；论文选择 5 个 stale steps，使 AE 早期一半步骤与 VLM 阶段重叠。

RTX 4090 +  $\pi_0$ : V-AEFusion 102 ms  $\rightarrow$  77 ms, 1.3×, 平均 SR 仅降 0.3 个百分点。

Finding 6: 相同优化在不同硬件上的收益可能完全不同

Table 9. V-AEFusion Performance Comparison across Different Hardware Platforms.

| Method     | 4090 (ms) | Orin (ms) | Thor (ms) | B60 (ms) | 310P (ms) |
|------------|-----------|-----------|-----------|----------|-----------|
| Baseline   | 102       | 920       | 246       | 306      | 818       |
| V-AEFusion | 77        | 806       | 236       | 292      | 820       |
| Speedup    | 1.32×     | 1.14×     | 1.04×     | 1.05×    | 1.0×      |

为什么 V-AEFusion 在 310P 上几乎无效?

因为硬件本身已经比较“吃满”了。  
VLM 和 Action Expert 同时跑时，会抢同一块硬件的计算和内存资源。  
不是“能并行”就一定“能加速”，还要看硬件有没有余量。

$$\text{Speedup} = f(\text{model}, \text{hardware}, \text{runtime})$$

硬件感知是必要条件：同一个优化方法不能假设跨平台等效。

|   |         |
|---|---------|
| 1 | 研究背景    |
| 2 | 模型-硬件配对 |
| 3 | 计算特征分析  |
| 4 | 加速方法    |
| 5 | 小结      |

## 本文贡献与结论

1

异构 VLA-XPU  
Leaderboard

用 Cost / Energy / Time 评价真实机器人部署，不只看 GPU latency。

2

双阶段瓶颈表征

VLM compute-bound, Action Expert memory-bound; 解释硬件利用率低和端到端频率差异。

3

面向结构的加速

DP-Cache 减少 diffusion 冗余; V-AEFusion 通过异步流水隐藏 VLM/AE 串行延迟。

**不要只看参数量和峰值 FLOPS; 要基于 workload profiling 做模型—硬件联合选型与 phase-aware 加速。**

## 局限与思考

### 局限 1

深度 profiling 主要围绕  $\pi 0$  类双系统 VLA，未来模型结构改变后瓶颈可能迁移。

### 局限 2

DP-Cache 的 stable segment 目前依赖离线 profiling 固定窗口，尚未运行时自适应。

### 局限 3

V-AEFusion 假设相邻观测高度相似；高速动态场景或强 ego-motion 可能削弱效果。

### 思考

- 1.异构协同推理：让 GPU / NPU / XPU 分别执行更适合自己的 VLA 阶段，而不是整个模型固定跑在一块硬件上。
- 2.按任务需求动态选硬件：实时性要求高时用高性能 GPU，低负载或续航优先时切到低功耗 NPU。
- 3.动态调整 VLA 配置：根据场景难度切换大/小模型、精度、DP-Cache、stale steps 等策略。
- 4.做一个统一调度器：综合模型阶段、硬件负载、延迟、能耗和任务需求，实时决定“谁来跑、跑哪一部分”。